Manual Técnico

Interfaz Gráfica

La interfaz gráfica del programa se compone de diversos elementos diseñados para facilitar la interacción del usuario con el compilador Cup y Flex. A continuación, se describen detalladamente cada uno de los componentes de la interfaz.

Botones:

Archivo: Este botón proporciona opciones relacionadas con la gestión de archivos, como abrir y guardar archivos de código fuente.

Pestaña: Permite organizar el espacio de trabajo mediante la creación y gestión de pestañas para diferentes archivos de código.

Ejecutar: Inicia el proceso de compilación y ejecución del código fuente proporcionado en el área de entrada.

Reportes: Facilita la visualización de informes y resultados generados durante el proceso de compilación.

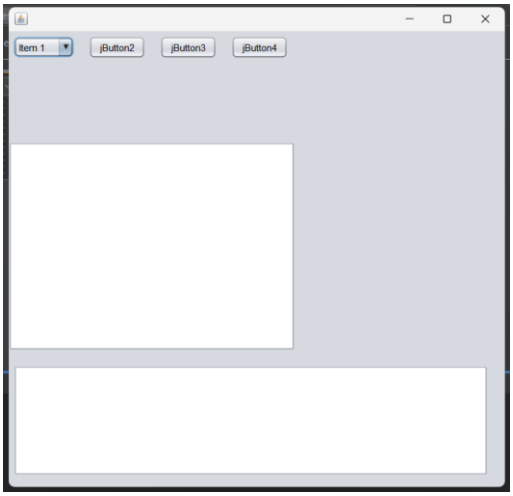
Áreas de Texto:

Área de Entrada de Código: Este espacio permite al usuario ingresar el código fuente escrito en lenguaje de programación especificado. Aquí se introducirán los programas escritos en Cup y Flex.

Área de Consola: Proporciona una salida interactiva para mensajes de estado, errores y resultados generados durante la compilación y ejecución del código.

Label para Gráficas:

Se incluye un label destinado a mostrar gráficas e informes visuales generados durante el proceso de compilación. Este componente puede ser de vital importancia para proporcionar una representación gráfica de resultados o datos relevantes.



Clase Principal (Main)

La clase principal OC1_Proyecto1 contiene la función main, que es el punto de entrada del programa.

Importaciones: Se importan las clases y paquetes necesarios, incluyendo Arbol, BufferedReader, StringReader, Scanner, Parser, Simbolo, y ArrayList.

Declaración de la Lista de Símbolos (Tabla): Se declara una lista (ArrayList) llamada Tabla para almacenar los símbolos.

Bloque try: Se inicia un bloque try para manejar posibles excepciones durante la ejecución del código.

Definición del Programa de Ejemplo (text): Se define un programa de ejemplo en un lenguaje específico.

Creación del Scanner y del Parser:

Se crea un objeto Scanner para analizar el texto proporcionado.

Se crea un objeto Parser que toma el resultado del Scanner y realiza el análisis sintáctico.

Obtención de la Raíz del Árbol Sintáctico Abstracto (AST): Se obtiene la raíz del AST al llamar al método parse del Parser.

Impresión de la Raíz del AST: Se imprime la representación en cadena de la raíz del AST en la consola.

Llamada a la Función recorrer: Se invoca la función recorrer en la raíz del árbol, la cual popula la tabla de símbolos Tabla con la información relevante.

Manejo de Excepciones (catch): En caso de que ocurra alguna excepción durante el análisis, se imprime la traza de la excepción.

Impresión de la Tabla de Símbolos (Tabla): Se imprime en la consola la información almacenada en la tabla de símbolos, mostrando el identificador (id) y el valor (contenido) de cada símbolo.



```
public class OC1_Proyecto1 {



    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {
        ArrayList<Simbolo> Tabla = new ArrayList();
        try {
            String text = "PROGRAM SUM(2,3) RES(2,3) MUL(2,3) DIV(2,3)  MOD(2,3) arr: double :: @carray <- [12, 2, 3] end; var:double:: numero <- 2

            Scanner scanner = new Scanner(new BufferedReader(new StringReader(text)));
            Parser parser = new Parser(scanner);
            Arbol raiz = (Arbol) parser.parse().value;

            System.out.println(raiz);
            raiz.recorrer(raiz, Tabla);



        } catch (Exception e) {
          //  System.out.println(e);
          e.printStackTrace();
        }for(Simbolo s : Tabla){
              System.out.println("id: "+s.id + " Valor: "+s.contenido);
            }
    }

}
```

Flex (Lexer)

El código proporcionado es un archivo Flex (Lexer) que se encarga de analizar el texto de entrada y generar tokens para ser consumidos por el parser.

```
%class Scanner
// Nombre de la clase que genera JFlex
%public // Para tener acceso desde otros paquetes
%line // Para registrar las líneas
%char // Llevar un conteo de caracteres
%cup // Habilita la integración con Cup
%unicode // Reconocimiento de caracteres unicode
%ignorecase // Omite el case sensitive, mayúsculas y minúsculas son iguales
```

%class Scanner: Indica que la clase generada por JFlex se llamará Scanner.

%public: Permite el acceso a la clase desde otros paquetes.

%line: Habilita el seguimiento de las líneas de entrada.

%char: Realiza un seguimiento de los caracteres de entrada.

%cup: Habilita la integración con Cup.

%unicode: Permite el reconocimiento de caracteres Unicode.

%ignorecase: Ignora las diferencias de mayúsculas y minúsculas en las reglas.

Reglas de Expresiones Regulares:

```
"program" { Globales.tokens.add(new Token("program",yyline, yycolumn, yytext()));
 return new Symbol(sym.PROGRAM, yyline, yycolumn, yytext()); }
```

Define las reglas para reconocer y generar tokens para palabras clave como "program".

Reglas de Expresiones Regulares:

```
{ENTERO}    { return new Symbol(sym.ENTERO, yyline, yycolumn, yytext()); }
```

Genera un token para los enteros encontrados.

Reglas de Expresiones Regulares:

```
"char[]" { Globales.tokens.add(new Token("char[]",yyline, yycolumn, yytext())); return new Symbol(sym.TIPO, yyline, yycolumn, yytext()); }
```

Genera un token para el tipo de datos char[].

Manejo de Errores Léxicos:

```
   Globales.error.add(new Token("Error lexico",yyline, yycolumn, yytext()));
 System.out.println("Lexical error: "+yytext()+" linea: "+yyline+" columna: "+yycolumn);
    throw new RuntimeException("Error léxico en la entrada");
}
```

Cup (Parser)

Captura cualquier carácter no reconocido y lanza una excepción de error léxico.

Terminales y No Terminales:

```
terminal String ADD, MINUS, TIMES, DIV, ENTERO, DECIMAL, PRINT, CON, COL, ES, EXEC, dgraf,tgraf ,PARE;
terminal String VAR, END_VAR, PROGRAM, END_PROGRAM, ARR, AT, END_ARR, LBRACKET,IGUAL, RBRACKET, COMMA, DEST,TIPO,OP, PARDER, ID, FIN, MAS, MENOS,
terminal String CADENA;

non terminal Arbol list;
non terminal String variable;
non terminal Arbol list_var, grafica,sentencia, tipo_sentencia;
non terminal Arbol list_expresion, argumentos;
non terminal Arbol expresion, list_cod,list_arreglos,list_varA , tipo_arreglo;
non terminal cadena, imprimir, valor, variab;
```

Se definen los símbolos terminales (tokens) y no terminales que se utilizarán en las reglas de producción.

Precedencias y Asociatividades:

```
precedence left ADD, MINUS;
precedence left TIMES, DIV;
```

Se especifican las precedencias y asociatividades para los operadores aritméticos.

Producciones:

```
list ::= PROGRAM list_cod:list_c END_PROGRAM {...};
list_cod ::= list_cod:list list_expresion:list_cexp {...}
          | list_expresion:list_exp {...};
list_expresion ::= list_var:listVar {...}
               | list_arreglos:array {...}
               | expresion:exp {...}
               | cadena {...};
```

Define las reglas de producción para la gramática del lenguaje. Cada producción especifica cómo se deben combinar los no terminales y terminales para formar estructuras más grandes.

Producciones para las Sentencias Gráficas:

```
grafica ::= tgraf:$0 PARE:$1 tipo_sentencia:$2 EXEC:$3 tgraf:$4 END_VAR:$5 PYC:$6 PARDER:$7 END_VAR:$8 PYC:$9{:
Arbol list_varA = new Arbol ("grafica");
list_varA.añadirHijo(new Arbol($0));
list_varA.añadirHijo(new Arbol($1));
list_varA.añadirHijo($2);
list_varA.añadirHijo(new Arbol($3));
list_varA.añadirHijo(new Arbol($4));
list_varA.añadirHijo(new Arbol($5));
list_varA.añadirHijo(new Arbol($6));
list_varA.añadirHijo(new Arbol($7));
list_varA.añadirHijo(new Arbol($6));
list_varA.añadirHijo(new Arbol($7));
RESULT = list_varA;
:};


tipo_sentencia ::= tipo_sentencia:$0 sentencia:$1{:
Arbol list_varA = new Arbol ("tipo_sentencia");
list_varA.añadirHijo($0);
list_varA.añadirHijo($1);
RESULT = list_varA;
:}
| sentencia:$0{:
Arbol list_varA = new Arbol ("tipo_sentencia");
list_varA.añadirHijo($0);
RESULT = list_varA;
:}
;

sentencia ::= dgraf:$0 DEST:$1 DEST:$2 TIPO:$3 IGUAL:$4 CADENA:$5 END_VAR:$6 PYC:$7{:
Arbol list_varA = new Arbol ("sentencia");
list_varA.añadirHijo(new Arbol($0));
list_varA.añadirHijo(new Arbol($1));
list_varA.añadirHijo(new Arbol($2));
list_varA.añadirHijo(new Arbol($3));
list_varA.añadirHijo(new Arbol($4));
```

Clase Árbol:

Atributos:

hijos: Un ArrayList que almacena los nodos hijos del árbol.

lexema: Una cadena que representa el valor o la etiqueta del nodo.

resultado: Un objeto que almacena el resultado calculado durante el análisis semántico.

Constructores:

public Arbol(String lexema): Constructor que inicializa el árbol con un valor de lexema y crea un ArrayList para los hijos.

Métodos:

public void añadirHijo(Arbol hijo): Agrega un nodo hijo al árbol.

public void recorrer(Arbol raiz, ArrayList<Simbolo> Tabla): Un método que realiza un recorrido recursivo del árbol. Durante el recorrido, se calculan resultados específicos y se almacenan en el atributo resultado.

Lógica de Recorrido y Cálculos:

En el método recorrer, se verifica si el nodo actual corresponde a ciertos tipos de expresiones, como operaciones matemáticas, funciones estadísticas (Media, Mediana, Moda, Varianza, Max, Min), y asignaciones de variables.

Se realizan cálculos específicos para cada tipo de expresión y se almacenan los resultados en el atributo resultado.

Uso de Variables Globales:

El código parece utilizar una clase llamada Globales para almacenar variables globales y operar con ellas. Esto incluye el almacenamiento y recuperación de variables mediante el uso de un HashMap.

Gestión de Tabla de Símbolos:

Se utiliza un ArrayList llamado Tabla para almacenar símbolos (objetos de tipo Simbolo). Durante el análisis semántico, se agregan nuevos símbolos a esta tabla.

```
    raiz.resultado = raiz.hijos.get(0).lexema;
}else if (raiz.lexema == "expresion" ){
    if(raiz.hijos.get(0).lexema.equalsIgnoreCase("SUM(")){
        System.out.println("LLegamos");
       Double res = Double.parseDouble(raiz.hijos.get(1).resultado.toString()) + Double.parseDouble(raiz.hijos.get(3).
        System.out.println(res);
       raiz.resultado = res;
    }else if(raiz.hijos.get(0).lexema.equalsIgnoreCase("RES(")){
        System.out.println("LLegamos2");
       Double res = Double.parseDouble(raiz.hijos.get(1).resultado.toString()) - Double.parseDouble(raiz.hijos.get(3).
        System.out.println(res);
       raiz.resultado = res;
    }else if(raiz.hijos.get(0).lexema.equalsIgnoreCase("MUL(")){
       Double mul = Double.parseDouble(raiz.hijos.get(1).resultado.toString()) * Double.parseDouble(raiz.hijos.get(3).
       System.out.println(mul);
       raiz.resultado = mul;

    }else if(raiz.hijos.get(0).lexema.equalsIgnoreCase("DIV(")){
       Double div = Double.parseDouble(raiz.hijos.get(1).resultado.toString()) / Double.parseDouble(raiz.hijos.get(3).
       System.out.println(div);
       raiz.resultado = div;
    }else if(raiz.hijos.get(0).lexema.equalsIgnoreCase("MOD(")){
       Double mod = Double.parseDouble(raiz.hijos.get(1).resultado.toString()) % Double.parseDouble(raiz.hijos.get(3).
       System.out.println(mod);
       raiz.resultado = mod;
    } else if (raiz.hijos.get(0).lexema.equalsIgnoreCase("Media(") ) {
        System.out.println("Calculando Media");
        System.out.println("HOLA" + raiz.hijos.get(1).resultado);
        Arbol listaValoresArbol = raiz.hijos.get(1);
        System.out.println(listaValoresArbol.resultado);

        ArrayList<Object> listaValores = (ArrayList<Object>) listaValoresArbol.hijos.get(1).resultado;
```

Clase Globales

Atributos estáticos:

Variables: Un HashMap que almacena variables globales, donde la clave es el nombre de la variable y el valor es el contenido de la variable.

tokens: Un ArrayList que almacena tokens reconocidos durante el análisis léxico.

error: Un ArrayList que almacena tokens que representan errores durante el análisis léxico.

Métodos estáticos:

imprimirHashMap(): Muestra en la consola el contenido del HashMap de variables.

guardarVariables(String clave, Object valor): Agrega una variable al HashMap de variables globales.

generarHTMLTokens(ArrayList<Token> tokens, String archivoSalida): Genera un archivo HTML que contiene una tabla de tokens.

generarHTMLErrores(ArrayList<Token> errores, String archivoSalida): Genera un archivo HTML que contiene una tabla de errores.

Método privado:

escribirTabla(ArrayList<Token> lista, BufferedWriter writer, String titulo): Escribe una tabla HTML (lexema, token, fila, columna) basada en una lista de tokens o errores. Este método es utilizado por los métodos de generación de HTML.

Uso de la Clase Token:

La clase Token se utiliza para representar información sobre tokens, incluyendo lexema, token, fila y columna.

Generación de HTML:

Los métodos generarHTMLTokens y generarHTMLErrores se encargan de escribir archivos HTML que contienen tablas de tokens y errores, respectivamente.

Estructura básica del HTML:

En los métodos de generación de HTML, se escribe una estructura básica de HTML que incluye las etiquetas <html>, <head>, <title>, <body>, etc.

```
public class Globales {

    public static HashMap<String,Object> Variables = new HashMap<>();
    public static ArrayList<Token> tokens = new ArrayList<>();
    public static ArrayList<Token> error = new ArrayList<>();
      public static void imprimirHashMap() {
        for (Map.Entry<String, Object> entry : Variables.entrySet()) {
            String clave = entry.getKey();
            Object valor = entry.getValue();
            System.out.println("Clave: " + clave + ", Valor: " + valor);
        }
    }

    public static void guardarVariables(String clave, Object valor) {
        Variables.put(clave, valor);
    }

    public static void generarHTMLTokens(ArrayList<Token> tokens, String archivoSalida) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(archivoSalida))) {
            // Escribir la estructura básica del HTML
            writer.write("<html>\n<head>\n<title>Tabla de Tokens</title>\n</head>\n<body>\n");

            // Escribir la tabla de tokens
            escribirTabla(tokens, writer, "Tokens");

            // Cerrar la etiqueta body y html
            writer.write("</body>\n</html>");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public static void generarHTMLErrores(ArrayList<Token> errores, String archivoSalida) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(archivoSalida))) {
            // Escribir la estructura básica del HTML
            writer.write("<html>\n<head>\n<title>Tabla de Errores</title>\n</head>\n<body>\n");
```